Three Dials, One Budget: Per-Token Compute Allocation with MoRE

Giulianno Vollmer
Lernex
giulianno@lernex.net

[TODO: date]

Abstract

Language models spend the same computation on the word *the* as on the step of a proof where everything goes wrong. Conditional-compute architectures fix this, but each of them fixes it along exactly one axis: Mixture-of-Depths and Mixture-of-Recursions vary *how many* steps a token gets, adaptive-k Mixture-of-Experts designs vary *how wide* a single step is, and recent looped MoE models fix both and vary neither. We present **MoRE**, in which a single per-token compute budget is allocated jointly across three axes: recursion **depth** (how many passes through a weight-shared stack), expert **width** (how many experts at each pass), and expert **pathway** (which experts, re-decided every pass from the token's evolving state). All three decisions are learned end to end from task loss under shared budget objectives. There is no halting threshold to tune, because there is no halting threshold. We add **MoRE-RM**, a recurrent depth memory whose entries are typed by pass, anchor, expert coalition, routed k, and router confidence, and which is read by the representation path and the routers at the same time — the model remembers not just what it computed, but who computed it and how sure they were. On a ladder of 13 models trained on byte-identical data at [TODO: tokens] we isolate each axis at matched model FLOPs, including the control this literature keeps forgetting: a *learned* policy against a *random* one at the same mean budget. [TODO: headline result] We close with Metis-1.6 Praxis and Logos, two MoRE models pretrained on 1T tokens, as evidence that the architecture survives contact with a real training run.

1 Introduction

[TODO: Write last. The four beats, in this order:]

1. Tokens are not equally hard, and every model in production knows this and does nothing about it. Predicting the second half of *"San"*→*"Francisco"* gets the same forward pass as resolving a pronoun three paragraphs back.

2. Conditional compute answers this, but every deployed variant turns exactly one dial — and "harder" is not one-dimensional. A token can need *more steps* (a multi-step calculation), *more capacity per step* (a rare entity), or *different capacity* (a switch from parsing to verifying). Depth, width, and pathway are not substitutes for one another, and a model that can only buy one of them will buy the wrong one.

3. MoRE learns all three jointly, per token, out of one budget. The claim is composition and joint training, not any single axis. Adaptive depth has prior art. Adaptive k has prior art. We say so in Section 2 rather than making a reviewer say it for us.

4. We back it with an axis-isolating ladder at matched model FLOPs. The uncomfortable control — learned policy versus random policy at the same mean budget — is included, and reported whichever way it comes out.

Contributions.

- **MoRE-Core**: joint per-token allocation over recursion depth, expert width, and expert pathway, with the routing and budget objectives that make three discrete decisions trainable from task loss (Section 3).
- **MoRE-RM**: a route-typed recurrent depth memory, native to MoRE in the strict sense that its key space is the token's own pass and route history, read by the representation path and the routers simultaneously (Section 4).
- A 13-model ablation ladder that isolates each axis at matched model FLOPs, plus random-policy controls that ask whether the *learned* allocation does anything or whether any allocation with the same budget would do (Section 6).
- A systems account of dropless packed dynamic compute — monotonic active sets, packed layouts, variable-count collectives — showing the architecture runs at [TODO:]% MFU rather than only in principle (Section 8).
- Two production models, Metis-1.6 Praxis and Logos, pretrained on 1T tokens (Section 9).

2 Related work

[TODO: This section is adversarial review done in advance. Be maximally generous to prior art on each individual axis; the novelty claim is the composition, and it is stronger, not weaker, when the pieces are properly credited. Sober tone throughout — no jokes at anyone else's expense.]

Adaptive depth. Adaptive Computation Time [cite: act2016] and PonderNet [cite: pondernet2021] established learned halting for recurrent computation. Mixture-of-Depths [cite: mod2024] makes per-token layer skipping learnable in transformers. Mixture-of-Recursions [cite: mor2025] is the closest prior work on this axis: per-token adaptive recursion depth over a weight-shared block, trained from scratch with a recursion router. Ouro [cite: ouro2025] studies looped models at scale and reports that recursion improves the *manipulation* of knowledge rather than the amount stored — a finding we adopt as a design constraint rather than dispute (Section 11). **Difference**: these pair adaptive depth with a dense feed-forward block. Depth is the only dial.

Adaptive width. Adaptive per-token expert count is established prior art and we claim no part of it. "Harder Tasks Need More Experts" [cite: harder2024] states the thesis in its title; AdaMoE [cite: adamoe2024] realizes variable k through null experts; DynaMoE [cite: dynamoe2026] adds layer-wise adaptive capacity; ProbMoE [cite: probmoe2026] formulates routing as a distribution over

cardinality-constrained expert subsets; AnyExperts [cite: anyexperts2025] allocates on demand in the multimodal setting. Expert-choice routing [cite: expertchoice2022] attacks the same imbalance from the expert side. **Difference**: all operate in a single-pass stack. Width is the only dial, and it is decided once.

Recursion combined with sparse experts. This combination became active immediately before this work, and we position against it directly rather than by omission. LoopMoE [cite: loopmoe2026] is a block-recurrent MoE at 3B/9B that resolves weight-sharing symmetry with IterAdaLN, a modulation signal conditioned jointly on the iteration index and the token state, evaluated against vanilla MoE at identical total parameters and per-token FLOPs. The Loopie series [cite: loopies2026] trains 20B/A2B and 6B/A0.6B looped MoE models with reasoning post-training. "Tying the Loop" [cite: tyingtheloop2026] compares per-iteration adaptation strategies including depth-wise LoRA [cite: relaxedrecursive2024], low-rank iteration modulation, and MeSH [cite: mesh2025]. **Difference, stated precisely**: in all of these the loop count is a fixed hyperparameter and the expert count per iteration is fixed. Neither the depth nor the width of the computation is a function of the token. Our claim is that making both adaptive *and jointly budgeted*, with the pathway re-decided from the evolving state at every pass, is what produces the gain — and our ladder includes a fixed-loop fixed-k MoE row (Section 6, row 5) that reimplements exactly this design point in our backbone, so the comparison is internal and matched rather than cross-paper and hopeful.

Residual topology and local memory. We use, and do not claim, Hyper-Connections [cite: hyperconnections2024] in a manifold-constrained form (mHC), the Mamba-2 hybrid backbone [cite: mamba2_2024], n-gram conditional memory in the Engram line [cite: engram], and aux-loss-free expert balancing [cite: deepseekv3]. Section 5 states the one thing we did modify — each is conditioned on the pass index — and claims that delta and nothing more.

3 MoRE-Core

3.1 Setup

A MoRE model is a weight-shared stack F of L physical layers. Every token receives pass 1. After pass r, a continuation router decides per token whether that token halts or enters pass $r+1$. The active set is monotonic, $A_{r+1} \subseteq A_r$, and is physically packed at every pass. Depth is capped at $R_{\max}$ passes.

We fix the convention *depth* = *passes* = *recursions* +1, so depth 1 is the non-looped base case, matching [cite: ouro2025].

Each layer's feed-forward sublayer is a mixture of E routed experts plus one always-active shared expert, computed in a latent space of width $d_\ell < d$. At every pass, and independently at every layer, a router chooses both how many routed experts the token gets and which ones.

3.2 The three decisions

For token t at pass r with state $h_t^{(r)}$:

Depth. A continuation router produces

$$p_t^{(r)} = \sigma\Big(g_\theta\Big(h_t^{(r)},\, m_t^{(r)},\, \Delta_t^{(r)},\, \rho_t^{(r)}\Big) + \tau_t^{(r)} - \tfrac{1}{2}\Big),$$

where $m_t^{(r)}$ is retrieved depth memory (Section 4), $\Delta_t^{(r)} = h_t^{(r)} - h_t^{(r-1)}$ is the state difference, $\rho_t^{(r)}$ is route history, and $\tau_t^{(r)}$ is a bounded, *detached* innovation prior $\tanh(\|\Delta\|/\|h\|)$. The prior gives the router a sensible monotonic initialization — tokens whose state is still moving get another pass — while leaving the learned logits free to override it from task loss. [TODO: Ablate the prior. A reviewer will reasonably ask whether the depth policy is learned or whether the prior is quietly doing the work, and they should ask. This is a cheap eval-time ablation on a trained checkpoint: zero τ and re-measure the depth distribution.]

Width. A k-router emits logits over $\{k_{\min}, \ldots, k_{\max}\}$. Let $q_t^{(r)}$ be the softmax at temperature γ and $\hat{k}_t^{(r)} = \sum_j q_{t,j}^{(r)} k_j$ the expected count. The hard counts are assigned by ranking $\hat{k}$ within the batch under the maximum-entropy integer marginal whose mean is exactly $k^\star$. The hard count drives the packed dispatch exactly; easy/low-ranked tokens receive fewer experts and hard/high-ranked tokens receive more, while neither all-$k^\star$ nor an all-min/all-max collapse can satisfy the marginal. The routed contribution is scaled by a straight-through envelope

$$\frac{k_t^{(r)} + \Pi_{\mathcal{A}}\Big(\hat{k}^{(r)}\Big)_t - \mathrm{sg}\Big[\Pi_{\mathcal{A}}\Big(\hat{k}^{(r)}\Big)_t\Big]}{\mathrm{sg}[k_t^{(r)}]},$$

where $\Pi_{\mathcal{A}}(x)_t = \mathbf{1}[t \in \mathcal{A}](x_t - |\mathcal{A}|^{-1} \sum_{j \in \mathcal{A}} x_j)$ projects onto the active fixed-budget tangent. The envelope is identically one in the forward pass, but task credit can only *reallocate* width between tokens; its infeasible common mode cannot ask the exact-budget executor to spend more compute. This is what makes width a *learned* decision rather than a scheduled one.

Exact depth budget. At every continuation boundary, the model ranks the active tokens by $p_t^{(r)}$. The number that survive is fixed by the maximum-entropy integer depth marginal over $\{1, 2, 3\}$ with mean $d^\star = 2$. Thus the router learns *which* tokens deserve later passes while the batch budget fixes only *how many*. Random-depth and random-k controls use the identical marginals with random rankings, so learned-versus-random rows execute exactly the same hard compute. Depth uses the same tangent-projected straight-through credit. A boundary whose quota is zero or all active tokens carries no task gradient, because no allocation decision exists there.

For both exact-budget routers, the policy reads a stop-gradient copy of the shared representation. The policy heads and their route-feature projection still learn from task and calibration losses, while the backbone learns from the hard computation that actually ran rather than from a continuous relaxation the executor cannot realize. Model, depth-policy, and width-policy gradients are clipped independently at norm one.

Pathway. The expert router is re-evaluated at every pass from the updated state and current route features, so the selected coalition can change even when k does not. Shared physical weights therefore do not imply repeated identical computation: successive passes can specialize into stages. [TODO: This axis has the weakest support in the prior literature and the strongest story in ours. The expert-transition-matrix figure (Section 7) carries it; row 6 of the ladder proves it.]

3.3 Training three discrete decisions at once

$$\mathcal{L} = \mathcal{L}_{\text{LM}} + \lambda_k\big(\bar{\hat{k}} - k^\star\big)^2 + \lambda_d\big(\bar{\hat{d}} - d^\star\big)^2 + \lambda_{k,\text{cal}}\,\text{CE}(q,k) + \lambda_{d,\text{cal}}\,\text{BCE}(p,c) + \lambda_z\mathcal{L}_z - \lambda_H\mathcal{H}\Big[p^{(r)}\Big],$$

with $k^\star$, $d^\star$ the exact hard marginal means; the quadratic terms calibrate the soft surrogates but do not enforce the hard budget. Here k is the assigned expert count and c the assigned continue/halt target. Expert load balancing uses the aux-loss-free selection-bias update [cite: deepseekv3] rather than a balance loss, so the balancing objective is not fighting the width objective for control of the same router. [TODO: Report the calibration coefficients and a sensitivity sweep.]

Why one budget and not two. The current campaign fixes exact depth and width marginals separately. That proves learned allocation on both axes but does *not* by itself prove a joint trade between them. The depth–width correlation analysis in Section 7 is therefore a falsifier, not decoration; a true shared-cost controller remains required before the title's "one budget" claim can be stated without qualification.

4 MoRE-RM: route-typed recurrent depth memory

4.1 Definition

At each attention anchor and at the end of every completed pass, each active token writes a typed memory entry: a learned state representation together with its *route type* — pass and anchor identity, the weight-averaged expert-coalition embedding, routed k, expert-router entropy and confidence, and continuation confidence. At later passes each current residual stream attends over that token's own valid entries, masked per token and bounded by $R_{\max}$.

Retrieval has two simultaneous consumers. The **representation path** gates memory back into the residual streams before the mixer, so computation can recover, compare, verify, or revise earlier reasoning states. The **routing path** feeds the depth, width, and pathway heads alongside the current state and state-difference features. No fixed cosine threshold or hand-written halt rule decides depth, because none is needed once the routers can see what the earlier passes actually did.

4.2 Relation to MeSH and to multi-stream residuals

The nearest published mechanism is MeSH [cite: mesh2025], which replaces the overloaded hidden state of a recursive transformer with an explicit buffer of B slots governed by step-wise read/write routers. Given the surface similarity — both add memory to a recursive stack — the distinction is worth making precisely rather than gesturing at.

MeSH maintains B slots $\mathbf{m}_b \in \mathbb{R}^{L\times D}$ initialized with the embeddings in slot 0, and per-iteration routers $\mathbf{w}^{(t)}_{\text{write}} = \text{softmax}(\text{Linear}^{(t)}_{\text{write}}(\mathbf{h}^{(t)}))$ and likewise for reading, with unique parameters per iteration t. Writing is additive into the slots and reading is a weighted sum over them. Three structural consequences follow:

1. **MeSH is closer to our mHC path than to MoRE-RM.** A set of parallel per-position state slots, mixed by iteration-conditioned learned weights, is functionally a multi-stream

residual with pass-dependent mixing — which is what recursion-aware mHC [cite: hyperconnections2024] is (Section 5). We therefore cite MeSH as convergent with our residual path, and credit it as an independent derivation of that idea, rather than treating it as prior art for the memory.

2. **MeSH slots are untyped and carry no route information.** The buffer records no expert coalition, no routed k, no router entropy, no halting confidence — because in a fixed-loop dense-or-MoE-agnostic model none of those quantities exists. MoRE-RM entries are keyed by exactly those quantities, and the read is attention over a growing set of typed entries rather than a softmax mixture over a fixed slot count.

3. **MeSH memory has no routing consumer.** Its read reconstructs $\mathbf{h}^{(t+1)}$ and nothing else; recursion depth in MeSH is a fixed hyperparameter with no halting mechanism, and there is no width or pathway decision for a memory to inform. MoRE-RM's second consumer — the routers — is not an extra feature bolted on, it is the reason the memory is typed at all.

The honest summary: MeSH solves *undifferentiated computation across iterations*, and we solve that too, with mHC. MoRE-RM solves a different problem that only exists once depth and width are decisions — giving those decisions evidence about what the earlier passes did and how confident they were. A single-pass or fixed-loop model cannot adopt MoRE-RM unchanged, which is the criterion in Section 5 and the reason it is claimed here.

[TODO: Consider one empirical version of this argument: strip the route typing from MoRE-RM entries (state representation only, no coalition/entropy/confidence) and re-run. If typed and untyped entries perform identically, the honest conclusion is that we built a more expensive MeSH, and we should say so. Cheap enough to be worth knowing.]

5 Integrations we use but do not claim

We apply a single criterion consistently: *a mechanism belongs to the MoRE contribution if its inputs or key space are defined over passes and routes; if a single-pass, fixed-budget model could adopt it unchanged, it is an integration.*

Mechanism	Status	Basis
Depth / width / pathway routing + budget objectives	contribution	definitional
Route-typed recurrent depth memory	contribution (MoRE-RM)	keyed by pass and coalition
Manifold-constrained Hyper-Connections (mHC)	integration	[cite: hyperconnections2024]
Mamba-2 / attention hybrid backbone	integration	[cite: mamba2_2024]
n-gram conditional memory	integration	[cite: engram]
Aux-loss-free expert balancing	integration	[cite: deepseekv3]

Two integrations are modified rather than adopted, and we claim the modification only. The mHC controller receives a learned pass embedding and pass-specific biases, so pass 1 through pass $R_{\max}$ use different residual topologies without duplicating weights. The n-gram injection gates are pass-aware, so a retrieved static vector is not added unconditionally on every pass; the current recurrent

state decides whether to use it. Both deltas exist because the base technique had no notion of a pass, which is also why they are small.

We note the convergence: LoopMoE's IterAdaLN [cite: loopmoe2026] and MeSH's step-wise routers [cite: mesh2025] are independent solutions to the same weight-sharing symmetry problem that pass-conditioned mHC addresses. Three groups reaching for the same fix in the same year is reasonable evidence that the problem is real.

6 Experiments

6.1 Design

All ablation models share one backbone — the same hybrid mixer stack, mHC residual path, and n-gram memory — and differ only in the routing axes under test. Holding the backbone constant means our *baselines are stronger than their standard counterparts*, which makes the comparison conservative for MoRE and removes backbone quality as an explanation for any gap we find. We state this rather than let a reader assume "Dense" means a vanilla transformer, because it does not.

Every model sees the *identical token stream in identical order*, sampled proportionally from the Metis-1.6 1T mixture including its phase structure, so data order is not a confound for anyone. Fixed-depth rows use the measured mean depth of MoRE-Core; fixed-k rows use $k = 4$.

Every row also includes the same n-gram conditional memory: a learned lookup table indexed by recent token pairs and triples, following the Engram line [cite: engram]. Its 0.300B stored values and its injection path are held fixed across the primary ladder; they are part of the shared backbone, not a variable in the MoRE ablation. The stored-parameter figures include this table. A token retrieves only its keyed rows once and caches the result across recurrent passes, so there is no corresponding 0.300B of active model weights.

The table reports model parameters in billions as stored/active (S/A). "Active" is the physical model subnetwork used on one routed path, including the embedding and tied output head. Recurrent passes reuse the same physical weights, so repeated execution appears in GFLOP/token rather than multiplying the active count. Dense rows therefore have equal stored and active model parameters; sparse rows separate the full expert bank from the selected path.

#	Model	Recur.	Depth	Width k	Pathway	Mem.	Stored/Active	GF/tok M/E
1	Dense, FLOP-matched	–	1	–	dense	–	1.44B / 0.87B	7.09 / 7.62
2	Dense, parameter-matched	–	1	–	dense	–	1.81B / 1.24B	9.31 / 9.85
3	MoE, k=4	–	1	fixed	routed	–	1.81B / 0.279B	3.54 / 4.08
4	MoE, k=8	–	1	fixed	routed	–	1.81B / 0.336B	3.88 / 4.42
5	Fixed LoopMoE	yes	fixed 2	fixed	re-routed	–	1.81B / 0.279B	7.09 / 9.45
6	Loop, pathway frozen	yes	fixed 2	fixed	frozen	–	1.81B / 0.279B	7.09 / 9.45
7	MoR + dense FFN	yes	adaptive	–	dense	–	0.85B / 0.278B	7.08 / 8.15
8	MoR + fixed-k MoE	yes	adaptive	fixed	re-routed	–	1.81B / 0.279B	7.09 / 8.16
9	Fixed-depth, adaptive k	yes	fixed 2	adaptive	re-routed	–	1.81B / 0.279B	7.09 / 9.45
10	**MoRE-Core**	yes	adaptive	adaptive	re-routed	–	1.81B / 0.279B	7.09 / 8.16
11	**MoRE-RM**	yes	adaptive	adaptive	re-routed	yes	1.81B / 0.279B	7.09 / 8.16
12	Random-k control	yes	adaptive	random	re-routed	–	1.81B / 0.279B	7.09 / 8.16
13	Random-depth control	yes	random	adaptive	re-routed	–	1.81B / 0.279B	7.09 / 8.16

Active parameters are per pass; rows with mean depth 2 execute them roughly twice. GFLOP/token reports model work / issued hardware work. The scientific iso-FLOP comparison uses model work; issued work additionally includes the checkpoint replay required by each measured memory lane. Each row differs from a neighbour in exactly one factor.

- **Rows 5 vs 6** isolate pathway: identical FLOPs, identical depth and width, differing only in whether the expert coalition is re-decided after pass 1 or frozen. Row 5 reimplements the fixed-loop MoE design point [cite: loopmoe2026] in our backbone.
- **Rows 8, 9, 10** isolate depth and width against each other at matched model FLOPs. Row 9 issues more hardware work because its measured 40-APU lane uses layer checkpointing.
- **Rows 12, 13** hold the compute budget fixed and randomize the policy. To our knowledge this control is absent from the adaptive-compute literature, and it is the experiment most capable of embarrassing us, which is why it is in the required set rather than the appendix.
- **Row 7** isolates adaptive depth without sparse experts. It is matched to MoRE-Core on model FLOPs, not on total stored capacity; its lower stored count is a deliberate compute-match choice, while row 2 is the separate dense stored-parameter control.

Rows 1 and 5–13 are iso-model-FLOP by construction. Rows 2, 3, 4 are deliberate off-frontier reference points. Note that a wider single-pass MoE *cannot* be made iso-FLOP with a recursive model by raising k alone: going k=4 $\rightarrow$ 8 adds about 0.34 model GFLOP/token because expert GEMMs are a minority of the block, while a second pass adds roughly 3.55. Row 4 is a reference, not a match, and we say so before someone assumes we tried and failed.

6.2 Configuration

[TODO: Proxy manifest table: 2 physical layers (1 Mamba-2 + 1 attention), $d_{\text{model}} = 4096$, latent 2048, expert intermediate 1152, 72 routed + 1 shared, $k \in [1, 8]$ with $k^\star = 4$, $R_{\max} = 5$, $d^\star = 2$, sequence 4096, tokenizer 65,536. Fill in exact audited counts from the ablation manifest.]

[TODO: Learning rate: report the short per-archetype sweep and which value each row used. One LR across architectures of different active widths is not fair; tuning all thirteen is not affordable. State the compromise — a reviewer who sees it stated is satisfied; one who suspects it is not.]

6.3 Scaling

[TODO: Three controlled stored-parameter sizes — 0.61B, 1.81B, and 5.12B — $\times$ four archetypes (Dense, MoE, MoRE-Core, MoRE-RM), plus Praxis and Logos as larger scale demonstrations. Wave 2 trains each added point for 100B tokens; at 5.12B this gives the parameter-matched dense control about 19.5 tokens per stored parameter, without establishing convergence. XL holds $d_{\text{model}} = 4096$, latent width 2048, and expert intermediate width 1152 from Wave 1, but grows to six physical layers (three Mamba-2 and three attention) with 80 routed experts per layer. XS and Wave 1 remain two-layer models. Each four-way comparison shares one backbone, but the nearly logarithmic ladder varies size and physical depth across points; it is not a width-only scaling law. The question is whether the MoRE advantage holds, grows, or shrinks across these scales and shapes.]

6.4 Results

[TODO: Everything. Primary figure: quality vs executed FLOPs, all rows, one frontier plot. Secondary: quality vs tokens. Tertiary: quality vs wall-clock — the axis that matters in practice and the one MoRE is most at risk on.]

7 Analysis: does it actually adapt?

Loss curves show MoRE is not worse. This section is the evidence that it does the thing it says it does.

- **Allocation vs difficulty.** Depth and k distributions conditioned on token surprisal, frequency, and syntactic class.
- **Depth–width correlation.** If depth and k move together perfectly, one axis is redundant and the three-dial framing is oversold. We report the correlation whatever it says. [TODO: Run this first — it can change the paper's central claim, and it is better to find that out in July than in review.]
- **Expert transitions across passes.** Transition matrices between pass r and pass $r{+}1$ coalitions, testing the stage-specialization hypothesis (parse → manipulate → verify).
- **Halt calibration.** Reliability diagram for continuation confidence.
- **Test-time budget dial.** The same checkpoint under continuation caps $1..R_{\max}$ and forced uniform depths $1..R_{\max}$, trained with stochastic depth budgets so every cap is a supported operating point rather than an out-of-distribution override.
- **Oracle gap.** On a small evaluation subset, exhaustively search the best per-token depth and report what fraction of the oracle improvement the learned policy captures.

8 Systems: dropless packed dynamic compute

Adaptive-compute papers are usually rejected on "does this actually run fast?" rather than on quality, and rightly so; a FLOP you cannot schedule is a FLOP you did not save. This section is a differentiator, not an appendix.

[TODO: Cover: monotonic active sets and per-pass packing; dropless routing with variable-count collectives; the three quantities that must never be conflated (training-data token dropout: none; MoE overflow drop rate: zero; learned continuation halting: intended and load-bearing); FP8 role assignment and the BF16 parity lane; measured MFU and tokens/second; the cost of dynamic control flow against a static-shape baseline.]

9 Scale demonstration: Metis-1.6

This section is a demonstration, not a controlled comparison, and we would rather say that in the first sentence than have it inferred from a footnote.

Metis-1.6 Praxis and Metis-1.6 Logos are MoRE models pretrained on the same immutable 1T-token release with identical phase boundaries, tokenizer, and curriculum, differing only in scale. [TODO: parameter and active-parameter audits]

Comparison to published models. Any comparison across laboratories differs in data, token budget, and post-training, so we report total parameters, active parameters per token, and training tokens beside every score, and we plot quality against active FLOPs per token rather than presenting a ranked table.

We state one prediction in advance: at a 1T-token budget, MoRE models should be *competitive on reasoning and manipulation benchmarks and behind on closed-book knowledge benchmarks* relative to models trained on an order of magnitude more tokens. This follows directly from the depth-versus-knowledge finding in [cite: ouro2025] — recursion buys manipulation, tokens buy knowledge — and we would rather register it as a prediction of the architecture now than offer it as an explanation later.

10 Claims, evidence, and falsifiers

Claim	Evidence	Falsified if
Depth adaptivity beats a matched fixed depth	rows 9 vs 10 at iso-model-FLOP	row 9 $\geq$ row 10
Width adaptivity beats a matched fixed k	rows 8 vs 10 at iso-model-FLOP	row 8 $\geq$ row 10
Per-pass re-routing matters	rows 5 vs 6	no gap at iso-FLOP
The learned policy beats a random one at the same budget	rows 12, 13 vs 10	no gap
Depth and width are distinct axes	correlation analysis	near-perfect correlation
Later passes do useful work	per-pass quality gain	gain vanishes after pass 2
Route memory helps	row 11 vs 10	no gap, or later experts starve
Route *typing* is what helps	typed vs untyped entries	no gap (we built a costly MeSH)
Stage specialization is real	transition matrices	coalitions are pass-invariant

These were written before the runs, and we report against them either way.

11 Limitations

- **Ablation scale.** [TODO: state tokens/parameter per row]. The controlled ladder now reaches 5.12B stored parameters, but conclusions about relative architecture quality still do not automatically transfer to frontier scale. The Praxis and Logos demonstrations partially address this and do not close it, and anyone claiming otherwise about their own ablations is guessing too.

- **Recursion does not buy knowledge** [cite: ouro2025]. MoRE targets manipulation. Closed-book factual benchmarks are not where this architecture is expected to win, and we do not report them as though they were.
- **Wall-clock is not FLOPs.** Dynamic routing costs launch overhead, compaction, and variable collectives. We report wall-clock next to FLOPs even where the comparison is less flattering.
- **Budget coefficients are choices.** Two quadratic penalties on a shared budget is the most tunable part of the design. [TODO: sensitivity sweep]
- **The ladder shares a non-standard backbone.** Conclusions are about the routing axes on that backbone, and we have not shown they transfer to a vanilla transformer stack.
- **Three dials mean three ways to be wrong.** Every axis we added is another policy that can fail to learn, and the analysis section exists because we did not want to find that out from a reader.

12 Conclusion

[TODO:]

References